

AWS Secure Web Server Troubleshooting Report

Project Title

AWS Web Server Deployment with VPC, EC2, Nginx, and Security Best Practices

Student Name

Eniola Dankuwo

Report Title

Troubleshooting Report

Deployment Date

June 2026

1. Introduction

This troubleshooting report documents the main issues encountered during the deployment of a secure cloud-based web server on AWS.

The project involved creating a custom VPC, public subnet, Internet Gateway, route table, EC2 instance, security group, SSH access, and Nginx web server.

The focus of this report is to explain the problems encountered, identify their causes, and document the solutions applied.

2. Troubleshooting Summary

Issue	Problem	Solution
Issue 1	SSH connection timed out when trying to access the EC2 instance	Updated the security group SSH rule to allow port 22 from My IP
Issue 2	A subnet may be named public but still not have internet access	Configured a route table with 0.0.0.0/0 pointing to the Internet Gateway and associated it with the subnet
Issue 3	EC2 public IPv4 address can change after stopping and starting the instance	Checked and used the current public IPv4 address before SSH and browser testing

Issue 1: SSH Connection Timed Out

Problem

When I tried to connect to the EC2 instance from my local terminal using SSH, the connection timed out.

The command used was:

```
ssh -i cloud-webserver-key.pem ubuntu@13.62.127.173
```

The error message was:

```
ssh: connect to host 13.62.127.173 port 22: Operation timed out
```

This meant that my terminal was unable to reach the EC2 instance through port 22.

This was not a private key issue. If the key pair or username had been wrong, the error would likely have been:

```
Permission denied (publickey)
```

Because the error was a timeout, I understood that the problem was most likely related to network access or firewall rules.

Cause

The cause of the issue was the security group attached to the EC2 instance.

The inbound SSH rule was not allowing traffic from my current public IP address.

The EC2 instance was running, and the public IPv4 address was available, but the security group did not permit my machine to connect through port 22.

This likely happened because I stopped the project and continued later. My current public IP address had changed, so the previous SSH rule no longer matched my current network.

The security group itself did not reset. The allowed source IP simply no longer matched my current public IP address.

Solution

To solve the issue, I updated the SSH inbound rule in the security group.

I changed the SSH source to:

My IP

This allowed only my current public IP address to access the EC2 instance through SSH.

Steps Taken to Solve the Problem

1. I opened the AWS Management Console.
2. I went to the EC2 dashboard.
3. I selected the running EC2 instance.
4. I clicked the **Security** tab.
5. I opened the security group attached to the instance.
6. I clicked **Edit inbound rules**.
7. I located the SSH rule for port **22**.
8. I changed the source to **My IP**.
9. I saved the updated inbound rule.
10. I returned to my local terminal.
11. I retried the SSH command.
12. The SSH connection worked successfully.

Final SSH Rule

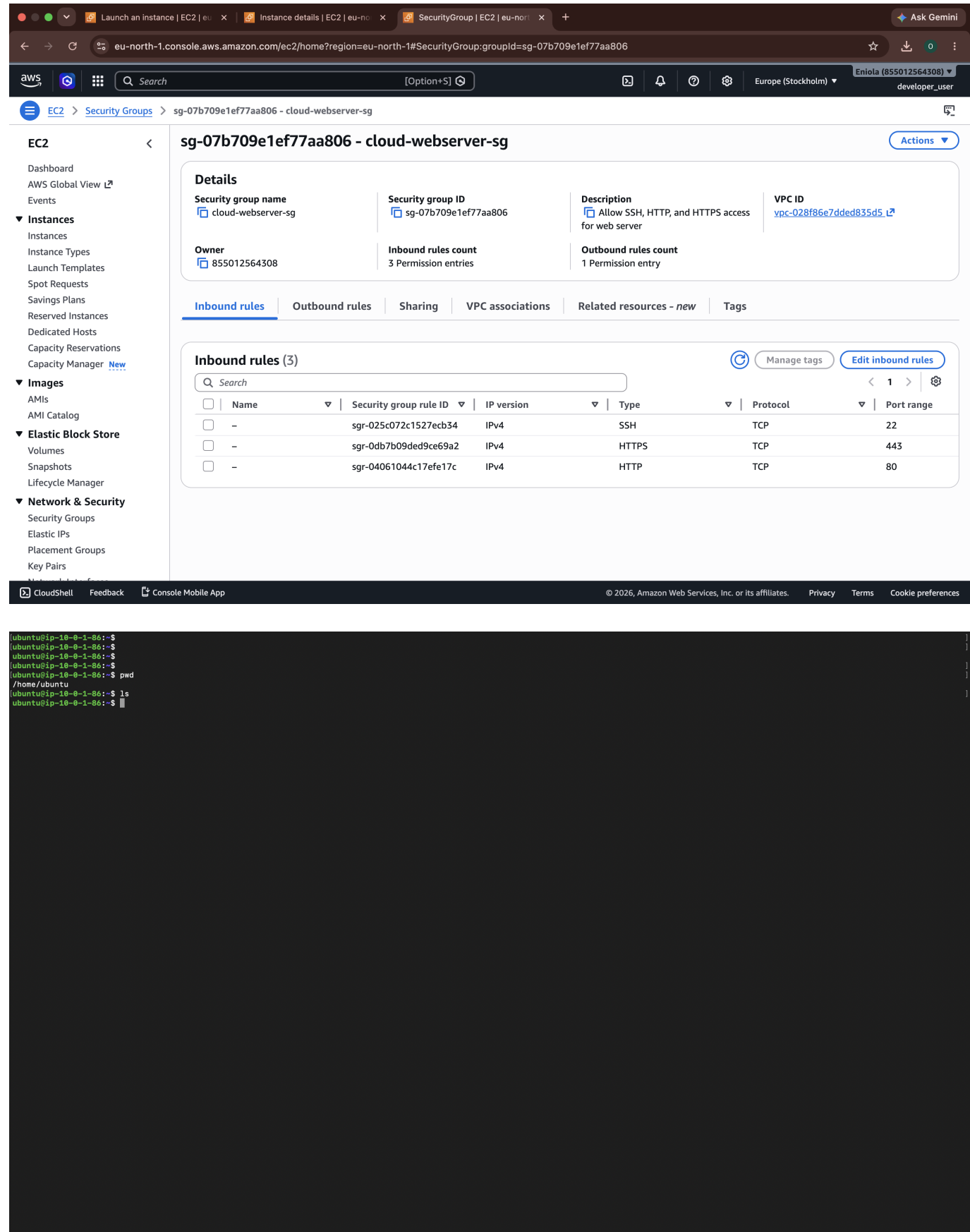
Type	Protocol	Port	Source	Purpose
SSH	TCP	22	My IP	Allows secure remote access only from my current IP address

Result

After updating the SSH source to **My IP**, I was able to connect successfully to the EC2 instance.

The terminal prompt changed to the Ubuntu EC2 instance prompt, confirming that I was inside the server.

Screenshot Reference



Lesson Learned

I learned that SSH access to an EC2 instance depends on more than the private key.

For SSH to work, the following must be correct:

- The EC2 instance must be running.
- The correct public IPv4 address must be used.
- The correct username must be used.
- The correct private key must be used.
- The security group must allow inbound SSH traffic on port 22.
- The SSH source must match the current public IP address.

I also learned that restricting SSH to **My IP** is more secure than opening SSH to the entire internet.

Issue 2: Public Subnet Requires Correct Route Table Configuration

Problem

During the VPC setup, it was important to confirm that the subnet was truly public.

A common mistake is to assume that a subnet becomes public simply because it is named something like:

```
cloud-webserver-public-subnet
```

However, a subnet name is only a label. It does not control network behavior.

If a subnet is not properly associated with a route table that points to an Internet Gateway, resources inside it will not have proper internet access.

Cause

The cause of this issue is misunderstanding what makes a subnet public.

A subnet becomes public only when its associated route table has a default route to an Internet Gateway.

The required route is:

```
0.0.0.0/0 -> Internet Gateway
```

The route **0.0.0.0/0** represents traffic going to the internet. The Internet Gateway provides the path between the VPC and the public internet.

Without this route, an EC2 instance in the subnet may exist, but it will not behave like a public server.

Solution

To make the subnet public, I created and configured a custom route table.

I added a default route that sends internet-bound traffic to the Internet Gateway. I also associated the route table with the public subnet.

Steps Taken to Solve the Problem

1. I opened the AWS Management Console.
 2. I searched for **VPC**.
 3. I opened the VPC dashboard.
 4. I clicked **Route tables**.
 5. I created a route table named **cloud-webserver-public-rt**.
 6. I selected my custom VPC named **cloud-webserver-vpc**.
 7. I opened the **Routes** tab.
 8. I clicked **Edit routes**.
 9. I added a route with destination **0.0.0.0/0**.
 10. I selected the Internet Gateway **cloud-webserver-igw** as the target.
 11. I saved the route.
 12. I opened the **Subnet associations** tab.
 13. I clicked **Edit subnet associations**.
 14. I selected **cloud-webserver-public-subnet**.
 15. I saved the association.
-

Final Route Table Configuration

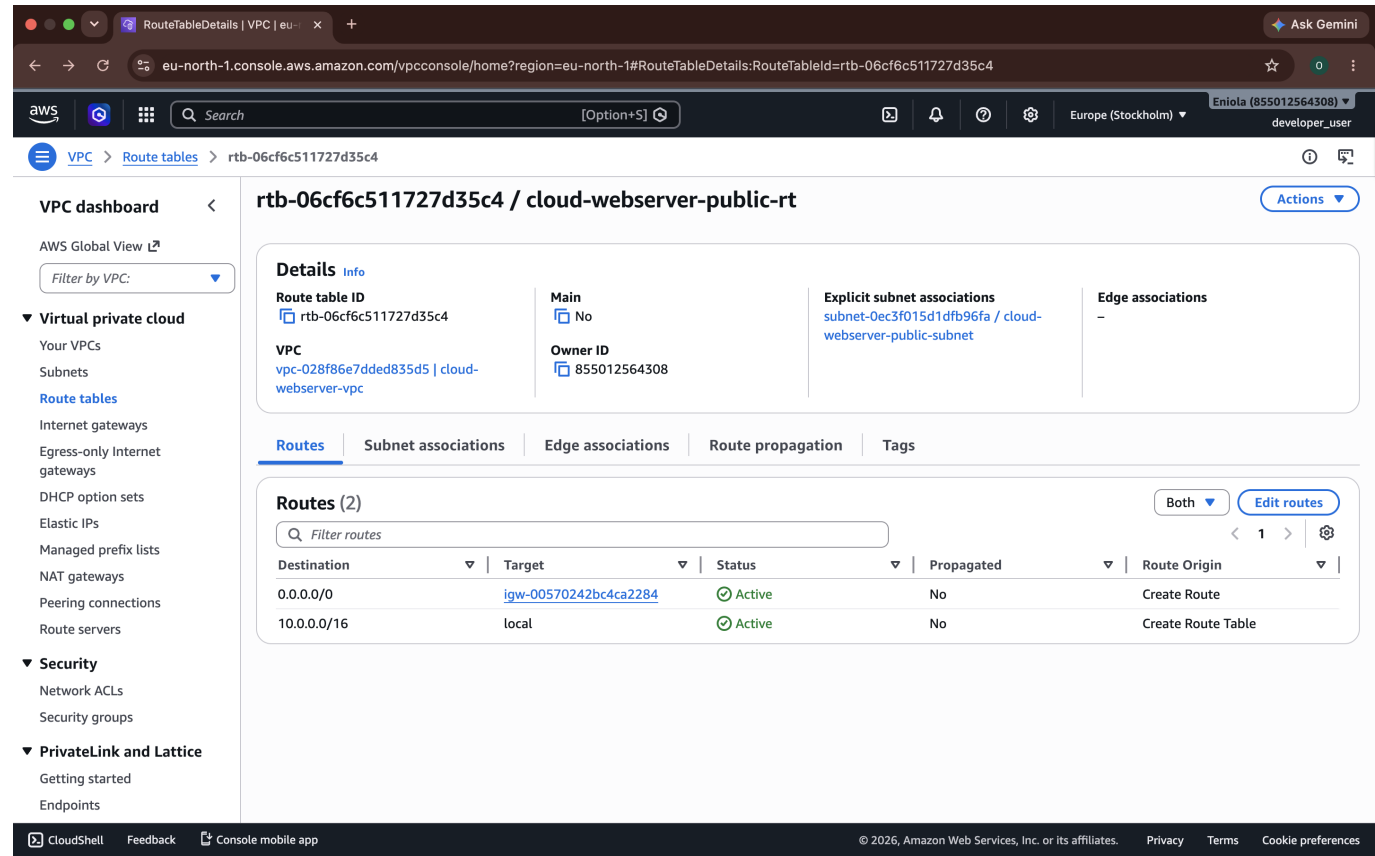
Destination	Target	Meaning
10.0.0.0/16	Local	Allows communication within the VPC
0.0.0.0/0	Internet Gateway	Sends internet-bound traffic to the Internet Gateway

Result

After adding the default route and associating the route table with the public subnet, the subnet became a functional public subnet.

This allowed the EC2 instance inside the subnet to support internet-facing web server access when combined with a public IP address and the correct security group rules.

Screenshot Reference



Lesson Learned

I learned that a subnet is not public because of its name.

A public subnet requires:

- A VPC
- A subnet
- An Internet Gateway attached to the VPC
- A route table with `0.0.0.0/0` pointing to the Internet Gateway
- A route table association with the subnet

This helped me understand how route tables control traffic flow inside a VPC.

Issue 3: EC2 Public IPv4 Address Can Change After Stop and Start

Problem

After pausing the project and stopping the EC2 instance, I needed to continue later.

When an EC2 instance is stopped and started again, the public IPv4 address can change if the instance does not have an Elastic IP address attached.

This matters because the public IPv4 address is used for:

- SSH connection
- Browser testing
- Final website URL
- Project documentation

If I used an old public IPv4 address, SSH or browser access could fail.

Cause

The EC2 instance was using an automatically assigned public IPv4 address.

Automatically assigned public IPv4 addresses are temporary. When the instance is stopped and started again, AWS may release the old public IP and assign a new one.

This happens unless an Elastic IP address is allocated and attached to the instance.

Solution

Before reconnecting to the instance or testing the website, I checked the EC2 dashboard and copied the current public IPv4 address.

I then used the current public IPv4 address for SSH and browser testing.

Steps Taken to Solve the Problem

1. I opened the AWS Management Console.
 2. I went to the EC2 dashboard.
 3. I selected my EC2 instance.
 4. I started the instance.
 5. I waited until the instance state changed to **Running**.
 6. I waited for the status checks to pass.
 7. I copied the current public IPv4 address from the instance details.
 8. I used the current public IPv4 address in my SSH command.
 9. I used the same current public IPv4 address to test the website in my browser.
-

Example SSH Command Format

```
ssh -i cloud-webserver-key.pem ubuntu@13.62.127.173
```

Result

By using the current public IPv4 address from the EC2 dashboard, I avoided connecting to an outdated IP address.

This ensured that SSH and browser testing were performed against the correct running EC2 instance.

Lesson Learned

I learned that a public IPv4 address assigned automatically to an EC2 instance is not permanent.

For a real production environment, I would use one of the following:

- Elastic IP address
- Domain name
- Load balancer
- CloudFront distribution

For this mini project, checking the current public IPv4 address before connecting was enough.

Final Troubleshooting Reflection

The troubleshooting process helped me understand that cloud problems should be investigated layer by layer.

The main layers I checked were:

Layer	What I Checked
Compute	EC2 instance state
Network	VPC, subnet, Internet Gateway, route table
Security	Security group inbound rules
Access	SSH key, username, and public IPv4 address
Web	Nginx service and browser access

The most important lesson I learned is that a cloud server depends on many connected services. Even if the EC2 instance is running, access can still fail if routing, security group rules, or public IP addressing are not correct.

Conclusion

This troubleshooting report documents three important issues encountered during the AWS web server deployment project.

The most practical issue was the SSH timeout caused by the security group not allowing access from my current IP address. Fixing it helped me understand how security groups protect EC2 instances.

The second issue helped me understand that public subnet behavior depends on route table configuration, not the subnet name.

The third issue helped me understand that automatically assigned public IPv4 addresses can change after stopping and starting an EC2 instance.

Overall, these troubleshooting experiences improved my understanding of AWS networking, EC2 access, public subnet design, and secure cloud deployment.

Author

Eniola Dankuwo Cloud/DevOps Engineering Track

GitHub: <https://github.com/Outlaw-Gypsy> **LinkedIn:**

<https://www.linkedin.com/in/dankuwo-eniola-124a4a19a/> **Email:**

enioladankuwo@gmail.com